

Event-Driven Workflow Management System

University of Messina

Department/course: Software Engineering course; exact department and course title were not provided in the repository evidence

Project: Event-Driven Workflow Management System

Reference workflow: Purchase Request Approval

Student: Yermek Aubayev

Matricola: 551098

Professor: name not provided in the repository evidence

Academic year: 2025/2026

Submission date: 21 August 2026

This report is intentionally evidence-based. Missing administrative information is marked as unavailable rather than inferred. The professor approved the Workflow Management System direction and accepted orchestration and choreography as the two complex functionalities. Purchase Request Approval is the final reference scenario selected to make that direction understandable; this report does not claim separate professor approval of that scenario or every final design decision.

Table of contents

1. Executive summary
2. Introduction
3. Real-world problem and system boundary
 - Target organization and actors
 - Before and after
 - Useful result and explicit boundary
4. Objectives, scope, and effort
 - Must Have scope
 - Exclusions
 - One-student, 100-hour justification
5. Requirements engineering
 - Stakeholder goals and functional requirements
 - Non-functional requirements
 - Constraints, acceptance criteria, and traceability
6. Development process and project control
 - Solo incremental method
 - Four truthful increments
 - Configuration/change management and risks
7. Evolution history
8. Product description
9. Reference workflow behavior
10. Architecture
11. Workflow Designer
12. Custom algorithms and invariants
13. Orchestration and choreography comparison
14. Verification and evidence
15. Quality evaluation
16. Reuse disclosure
17. Limitations and future work
18. Conclusion
19. References
20. Appendices
 - Requirement-to-implementation/test traceability
 - API summary
 - Defense evidence checklist
 - Final claim ledger

Executive summary

This individual Software Engineering project implements a bounded Event-Driven Workflow Management System for a small organization. The reference process is Purchase Request Approval. A requester submits structured purchase information; the system validates the request, creates a persistent approval work item, waits for a manager decision, resumes the same workflow run, creates an internal Purchase Authorization after approval, records an Internal Notification, and retains an ordered audit history. Negative outcomes are explicit: invalid input stops before human approval, rejection prevents authorization, temporary authorization failure can be retried within a configured bound, and exhausted failure leads to manual action.

The project demonstrates two control strategies over identical business semantics. Orchestration uses a central application loop. Choreography uses a synchronous in-process EventBus with a temporary handler scoped to one active run. Both use the same immutable workflow revision, task executors, retry policy, transition resolver, SQLite persistence, approval lifecycle, and business effects. The choreography claim is deliberately narrow: it compares event-reaction control ownership inside one modular application. It does not claim distributed services, durable messaging, asynchronous delivery, or exactly-once transport.

The principal custom engineering work is the workflow core: deterministic graph validation, transition precedence, bounded retry before routing, persistent versioned cursor, run isolation, immutable revisions, human wait/resume, idempotent decision handling with conflict detection, ordered audit trace, and restart recovery. A bounded Workflow Designer exposes this core safely through four predefined task types and three transition conditions while prohibiting arbitrary executable logic.

Verification is based on source inspection, connected API journeys, domain and application tests, persistence tests, deployment checks, and real-process restart tests. The final unchanged repository suite reports **98 passed**. The report distinguishes executed evidence from inspected design and documented limitations. No production performance, user research, security testing, deployment success, or stakeholder interviews are claimed.

Introduction

Workflow-management projects can become demonstrations of technical motion rather than useful systems. A screen may show nodes turning green, events being published, or retries occurring without answering why a user needs the product, who owns the next action, what persists while a person is unavailable,

and what useful result exists at the end. The design goal of this project is to connect those technical mechanisms to one concrete but limited organizational process.

Purchase Request Approval was chosen because it contains the engineering situations needed for the course objective without requiring external financial or procurement infrastructure. The process has structured input, deterministic validation, a real human wait, positive and negative decisions, an automatic effect after approval, retryable technical behavior, history, and a safe configuration boundary. It is understandable to an evaluator while preserving the two accepted complex functionalities: orchestration and choreography.

The system boundary is important. A Purchase Authorization is an internal record that the workflow approved a request. It is not a purchase order, budget reservation, supplier instruction, invoice, accounting entry, or payment. The Internal Notification is a stored application message, not an email. Demonstration failures are deterministic examination controls, not observations from a third-party service. These boundaries prevent the report from assigning capabilities to code that does not implement them.

This report is self-contained. It explains the problem, requirements, process, evolution, product, architecture, algorithms, comparison, evidence, quality, reuse, limitations, and defense checklist. UML figures are included at the point where they support an argument rather than as decoration.

Real-world problem and system boundary

Target organization and actors

The target is a small organization that needs to control repeatable internal approval processes but does not require a full enterprise process platform. In an informal approach, requests can arrive through messages or spreadsheets. Required fields vary, responsibility is unclear, duplicate decisions can occur, and a technical restart can make the state difficult to reconstruct. A process owner may also lack a safe way to adjust the sequence without editing source code.

Stakeholder or actor	Goal	Product response	Claim class
Requester	Submit complete purchase information and see the authoritative outcome	Structured form, request/run status, notification, audit history	Implemented and inspected
Approver	Receive persistent work with meaningful context and decide once	Inbox, work-item detail, approve/reject endpoint, idempotency/conflict rules	Implemented and tested
Process owner	Configure a bounded reusable process safely	Draft CRUD, validation, closed task catalog, immutable activation	Implemented and tested
Academic evaluator	Understand value and compare two control strategies	Business-first UI, shared scenarios, UML and paired tests	Implemented and inspected
Professor	Evaluate whether the final project satisfies academic expectations	Evidence report and defense script; acceptance remains external	Documented limitation

The logical roles are not security identities. There is no login, password, role assignment, or authorization middleware. Anyone who can access the local application can use its demonstration endpoints. That is acceptable only within the academic/local boundary and is a primary future-work item.

Before and after

Concern	Informal before-state	System after-state
Input	Inconsistent messages or spreadsheet rows	Required structured fields with deterministic validation
Ownership	Next actor may be unclear	Pending approval work item identifies the required action
Waiting	State depends on conversation context	Run and cursor persist in SQLite while no process action is active
Duplicate action	Repeated messages may create ambiguity	Identical decision is idempotent; conflicting decision is rejected
Temporary failure	Manual repetition may be uncontrolled	Retry is classified and bounded before graph routing
Process changes	Ad-hoc instructions	Validated draft and immutable activated revision
History	Timestamps/messages are fragmented	Ordered run-local audit trace and persisted effects
Restart	In-memory progress can disappear	Committed cursor and records permit same-run recovery

The comparison describes the problem framing and implemented response; it is not based on interviews or measurements at a deployed organization.

Useful result

The useful result is an authoritative, explainable state. A successful approved run reaches AUTHORIZED and has a Purchase Authorization plus Internal Notification. A rejected request reaches REJECTED and skips authorization. An invalid request reaches VALIDATION_FAILED before approval work exists. Exhausted authorization failure reaches NEEDS_MANUAL_ACTION. The run projection and ordered trace explain how the result was reached.

Figure 1 summarizes the actors and bounded use cases.

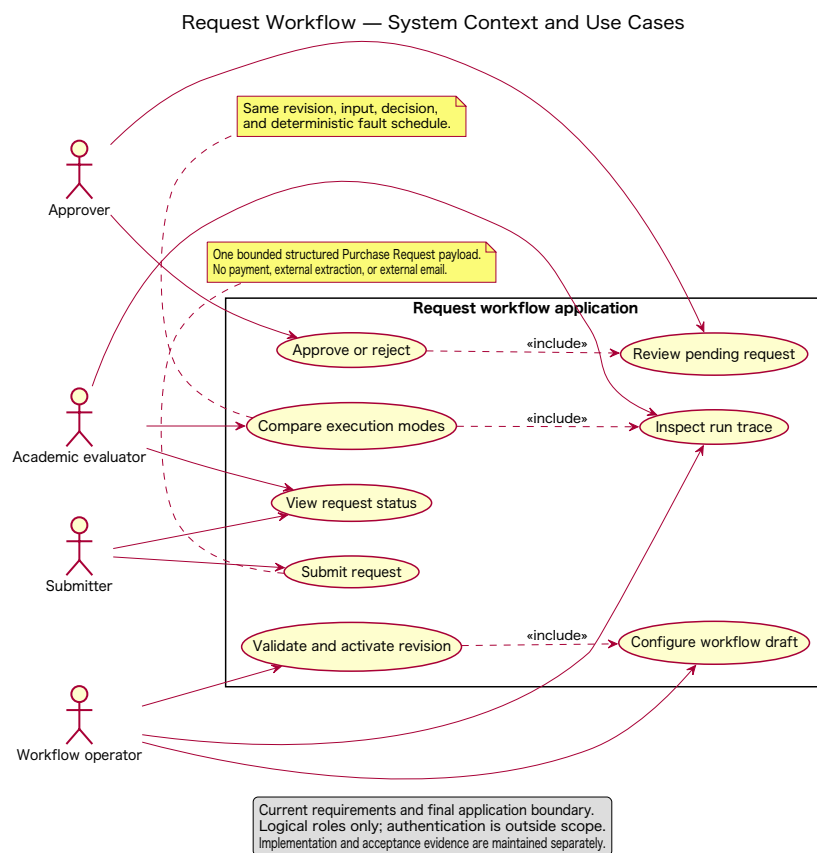


Figure 1. System context and bounded use cases.

Explicit boundary

The application does not place orders, reserve or transfer money, confirm budgets, contact suppliers, send external notifications, certify accounting treatment, perform fraud analysis, or integrate with ERP systems. It also does not authenticate actors, distribute execution across services, or promise production availability. These are exclusions, not partially implemented features.

Objectives, scope, and effort

Objectives and Must Have scope

The project objective is to demonstrate a configurable, persistent, event-driven workflow core through an understandable approval process. The Must Have scope was:

ID	Functional scope	Completion evidence
M1	Represent and validate a conditional acyclic workflow	Validator and constructor tests
M2	Activate immutable revisions and bind each run to one revision	Persistence/constructor tests
M3	Submit and validate structured Purchase Request data	Domain and connected journey tests
M4	Persist human approval work and resume the same run	Approval and restart tests
M5	Resolve success/failure transitions deterministically	Resolver tests
M6	Retry classified technical failures within a positive bound	Retry and step-service tests
M7	Execute equivalent semantics by orchestration and choreography	Paired scenario tests
M8	Persist attempts, effects, cursor, decisions, and ordered trace	Persistence and journey evidence
M9	Provide four understandable browser views	HTML inspection and local application smoke evidence
M10	Provide reproducible local and optional single-service container startup	Launcher and Compose verification

Exclusions

Authentication/RBAC, external integrations, distributed brokers, microservices, arbitrary user code, full BPMN, cycles, parallel fork/join, nested workflows, external notification delivery, production monitoring, high availability, and verified scalability were excluded. The exclusions protect the core learning objective and prevent one student from presenting framework configuration as custom workflow engineering.

One-student, 100-hour justification

The peer report used only as a structural reference represented three students and about 300 hours. This project represents one student and approximately 100 hours. It therefore uses one reference workflow, four task types, one deployable process, a compact browser UI, and no external service integration.

Activity	Hours
Problem framing, feedback analysis, and scope control	8
Requirements and traceability	12
Domain graph, resolver, and retry logic	16
Persistence, revision, cursor, and transaction design	15
Human approval, idempotency, and recovery	13
Orchestration/choreography implementation and comparison	12
API, UI, and bounded Workflow Designer	9
Automated verification and debugging	9
Documentation, UML, report, and defense reconciliation	6
Total	100

The table is a retrospective allocation by actual activity category. It does not invent work dates, meetings, standups, or time-sheet precision.

Requirements engineering

Stakeholder goals and functional requirements

Requirements were organized around the useful process outcome, then traced to implementation and tests. The detailed baseline is maintained in docs/requirements/requirements.md; this section summarizes the active behavior.

The requester must provide requester name, department, item or service, supplier, positive amount, three-letter uppercase currency, meaningful justification, and a valid non-past required date. Domain validation produces deterministic field-specific issues. Valid requests proceed to one human approval task. The approver can inspect context and submit approve or reject; rejection requires a reason. Approval permits Purchase Authorization, while rejection prevents it. Automatic steps persist attempts and use failure classification. Every run is isolated, records an ordered trace, and remains associated with the revision accepted at creation.

Non-functional requirements

The project avoids vague quality statements by pairing each metric with a threshold and evidence type.

Quality	Metric/threshold	Evidence
Determinism	Same revision/input/decision/fault schedule produces equivalent semantic outcome in both modes	Paired normalized scenario tests
Retry safety	Automatic executions never exceed configured positive <code>max_attempts</code>	Retry and step-service tests
Restart recovery	Waiting run reaches the same authorized result after real process recreation	Two Uvicorn restart tests
Isolation	No request, attempt, decision, effect, or trace crosses run identity	Sequential and controlled concurrent tests
Configuration safety	Invalid graph/task configuration cannot activate a revision	Constructor/validator negative tests
Trace completeness	Relevant committed state changes have ordered run-local observations	Exact trace and journey assertions
Deployment boundedness	Compose defines one application service and no broker	<code>docker compose config --quiet</code> and inventory tests
API inspectability	Current OpenAPI exposes request, approval, run, and workflow routes	OpenAPI smoke inspection
Maintainability	Business algorithms remain independent of FastAPI and SQLite types	Layer/source inspection and domain tests
Usability evidence	Four task-oriented headings and business context precede technical controls	HTML/manual inspection; no user-study claim

No response-time, throughput, concurrent-user, or uptime threshold is claimed because no credible performance environment or production workload was executed.

Constraints

The solution uses Python/FastAPI, SQLAlchemy, Pydantic, SQLite, a static browser client, pytest, and optional Docker. It remains a modular monolith. Workflow definitions are conditional DAGs. The task catalog is closed. The

EventBus is synchronous and in-process. Runtime databases are not versioned. These constraints are both feasibility controls and part of the system's claim boundary.

Acceptance criteria and traceability

Scenario	Input/action	Expected observable result	Automated evidence
Valid approval	Valid request, approve	AUTHORIZED; authorization and notification exist	Connected journey in both modes
Rejection	Valid request, reject with reason	REJECTED; no authorization	Connected journey in both modes
Invalid request	Invalid required field	VALIDATION_FAILED; no approval work item	Domain/journey tests
Retry then success	Valid approval, deterministic transient fault	Two authorization attempts, retry trace, AUTHORIZED	Retry scenario tests
Retry exhaustion	Valid approval, unavailable scenario	Bound reached, NEEDS_MANUAL_ACTION	Exhaustion scenario tests
Duplicate decision	Repeat identical decision	Existing result returned; no second advancement	Approval idempotency tests
Conflicting decision	Submit different second decision	Controlled conflict; state unchanged	Conflict tests
Restart/resume	Stop at wait, recreate process, decide	Same run resumes and reaches AUTHORIZED	Two real-Uvicorn tests

Traceability is bidirectional at the practical level: requirements point to components/tests, and the report's algorithm table points back to source and UML. Static source inspection supports structural claims but does not replace executed acceptance tests.

Development process and project control

Solo incremental method

The work used a solo incremental method influenced by Scrum concepts without pretending that one student formed a Scrum team. There were no claimed daily standups, stakeholder interviews, or formal sprint ceremonies. A product backlog, Definition of Done, risk register, evidence register, and change record were used to keep implementation and documentation aligned.

The Definition of Done required behavior, tests, source/doc consistency, clean dependency direction, reproducible startup, and evidence appropriate to the change. Git provided configuration management. A named branch and atomic commits made the migration and final reconciliation reviewable. CR-003 records the final product correction while preserving the evolution history.

Four truthful increments

Increment 1 - Generic workflow core

Goal: demonstrate a reusable conditional workflow engine. **Work/design:** define immutable task/transition types, graph rules, deterministic resolver, attempts, trace, and isolated runs. **Implementation:** domain modules and persistence-backed runtime. **Tests:** validator, resolver, retry, and isolation cases. **Review result:** the technical mechanisms worked, but the product value was difficult to explain through abstract nodes. **Retrospective:** the next increment needed a recognizable requester, decision, and useful result. **Increment result:** a reusable core worth preserving, not a complete product story.

Increment 2 - Human lifecycle and dual control

Goal: make time and control ownership explicit. **Work/design:** introduce persistent cursor state, approval work, same-run resume, orchestration, and run-scoped choreography. **Implementation:** approval services, coordinator, synchronous EventBus strategy, state-version checks, and ordered trace. **Tests:** waiting/resume, idempotency/conflict, paired modes, and handler cleanup. **Review result:** the two complex functionalities became testable under shared semantics. **Retrospective:** technical parity still needed a clearer business scenario. **Increment result:** durable workflow behavior within one process.

Increment 3 - Intermediate Invoice/PDF application

Goal: attach workflow behavior to a concrete approval artifact. **Work/design:** an Invoice Approval direction introduced uploaded PDF handling. **Implementation/review:** it made input tangible but expanded evidence around file safety, parsing, and storage. **Retrospective:** those concerns dominated the explanation and distracted from workflow management. **Increment result:** useful feedback and a historical intermediate direction, later removed from the active product.

Increment 4 - Purchase Request product and reconciliation

Goal: deliver an understandable structured reference workflow without file-processing distractions. **Work/design:** select Purchase Request Approval, retain the workflow core, add structured domain validation, internal authorization/notification effects, four browser views, closed designer, and comprehensive evidence. **Tests:** 98 final tests across domain, application, persistence, presentation, restart, and deployment. **Review result:** the active system and source behavior align with the reference workflow. **Retrospective:**

documentation required an additional evidence-focused pass because mechanical terminology changes had left contradictions. **Increment result:** the final implementation and reconciled report represented here.

Configuration and change management

The repository records requirements, ADRs, PlantUML sources, rendered SVGs, report source, build script, and final PDF. Runtime databases and caches are excluded from commits. Immutable runtime revisions provide product-level configuration management: a run never references a mutable draft, so later edits cannot reinterpret previous history.

Risks and mitigations

Risk	Impact	Mitigation/ evidence	Residual limitation
Product appears as a node simulator	High	Business-first UI, reference scenario, defense sequence	Evaluator acceptance remains external
Modes diverge	High	Shared services and paired normalized tests	Only one-process semantics demonstrated
Duplicate decision advances twice	High	uniqueness, idempotency, conflicts, tests	Multi-node database contention not evaluated
In-memory event is mistaken for durable state	High	persistent cursor and explicit claim boundary	No durable messaging
Retry loops or routes prematurely	High	positive bounds and retry-before-resolver tests	External backoff policies excluded
Run data contamination	High	run keys, constraints, versioning, isolation tests	Large-scale concurrency unmeasured
Unsafe workflow configuration	Medium	closed catalog and graph validation	No general BPMN features
Documentation overclaims evidence	High	claim ledger, stale-term audit, exact commands	Administrative title fields incomplete

Evolution history

The product evolution is evidence of requirements work rather than a reason to hide earlier decisions. The original generic demonstration emphasized engine behavior. Feedback showed that a viewer could not easily connect that behavior to an organizational benefit. The intermediate Invoice/PDF direction was a

reasonable attempt to add context, but it introduced a second center of gravity: file processing. The final correction removed those active concerns and selected structured Purchase Request Approval.

Core element	Final disposition	Reason
Graph validation	Preserved	Safe configuration remains fundamental
Transition resolver	Preserved	Mode-independent business routing
Bounded retry	Preserved and classified	Demonstrates controlled failure without graph cycles
Run isolation and trace	Preserved	Necessary for auditability and concurrent correctness
Orchestration	Preserved	Accepted complex functionality and clear central control
Choreography	Preserved and bounded	Accepted complex functionality; internal comparison only
Persistent cursor/recovery	Preserved and strengthened	Human waiting and restart require authoritative continuation
Invoice/PDF modules	Removed from active system	Distracted from workflow-management objective
Structured Purchase Request validation	Added	Makes business input explicit without file concerns
Purchase Authorization/Internal Notification	Added	Provide useful bounded outcomes
Four-view UI	Added/reframed	Leads with user work and result

CR-003 is the authoritative evolution record. Final academic acceptance remains the professor's decision.

Product description

The browser client is one page organized as four numbered views. **Submit Request** captures the business fields and starts a run. **Approver Inbox** lists pending work and shows request context before the decision. **Run Status / History** loads the authoritative projection and optional technical trace. **Workflow Designer** creates and validates bounded drafts and activates immutable revisions.

Execution mode and deterministic failure choice are inside a collapsed **Demonstration Controls** section. This placement is intentional. A normal user first sees the request and useful outcome; examination mechanisms remain available without being presented as everyday business functionality.

The API boundary mirrors these tasks. POST /api/requests creates a request and run. GET /api/approvals lists pending work; GET /api/approvals/{id} returns context; POST /api/approvals/{id}/decision commits a decision. GET /api/runs/{run_id} returns status/history. /api/workflows routes manage drafts, validation, activation, and revision reading.

Figure 2 shows the implemented component responsibilities.

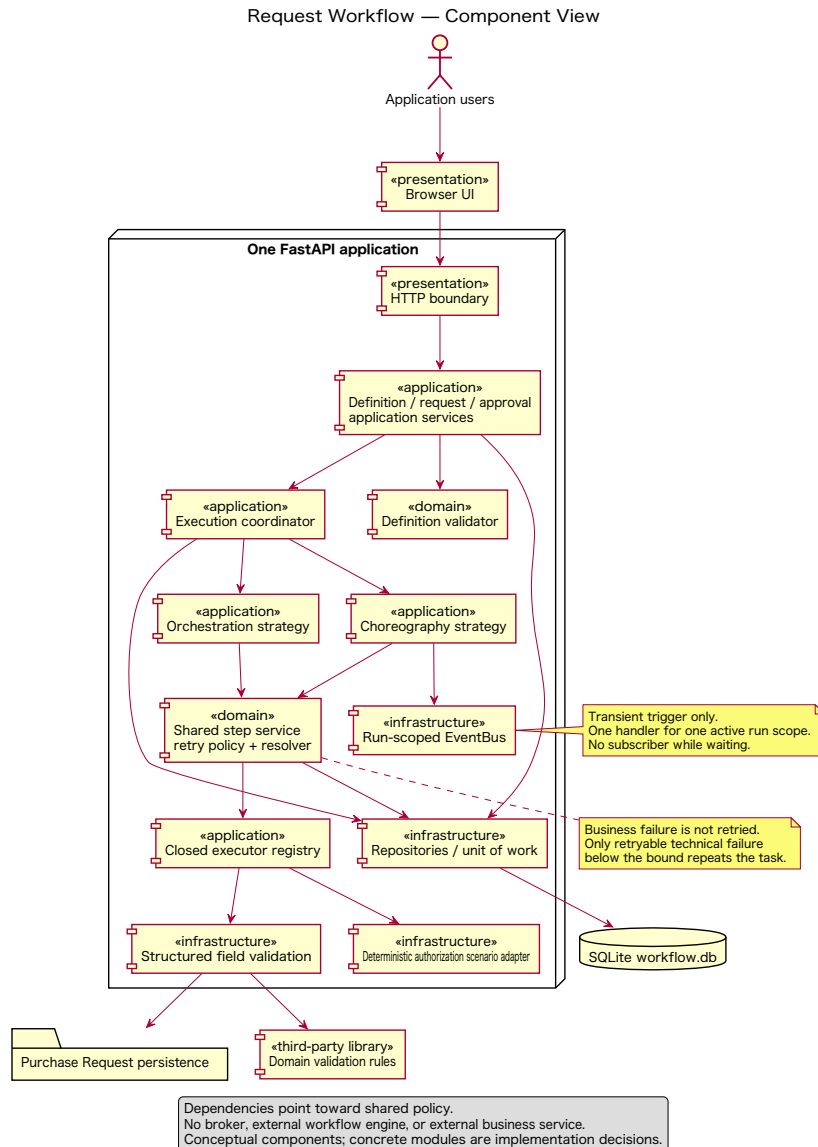


Figure 2. Modular-monolith component view.

Reference workflow behavior

The default workflow contains four tasks. REQUEST_VALIDATION is the start. On success it reaches HUMAN_APPROVAL; on validation failure it follows the configured negative route or terminates according to the revision. Approval success reaches PURCHASE_AUTHORIZATION; rejection supplies failure routing and skips

authorization. Successful authorization reaches `CREATE_NOTIFICATION`. Exhausted authorization failure applies the manual-action state before final failure routing.

Normal approval

A valid request creates run/cursor state. Validation records an automatic attempt and succeeds. Human approval creates a persistent work item and returns control. Approval commits the decision and resumes the cursor. Purchase Authorization succeeds, Internal Notification is recorded, and the run reaches successful terminal state with request state `AUTHORIZED`.

Rejection

The same submission and wait occur. The approver selects reject and supplies a reason. The decision maps to workflow failure semantics; authorization is not executed. The request remains `REJECTED`, notification records the controlled result, and trace entries show decision, resume, routing, and terminal behavior.

Invalid request

Domain validation reports one or more deterministic issues. The validation executor returns business failure, which is never retried. No approval work item is created. The configured route records notification/terminal behavior and the request becomes `VALIDATION_FAILED`.

Retry then success

The deterministic adapter causes the first authorization execution to return retryable technical failure. The attempt is committed. Because the count is below the bound, the retry policy keeps the cursor on the same task and records retry without selecting a transition. The next attempt succeeds, effects are applied, and normal success routing continues.

Retry exhaustion

The adapter returns retryable failure for every allowed authorization attempt. When the completed count reaches the bound, no further retry occurs. The business effect policy exposes `NEEDS_MANUAL_ACTION`, then final failure enters ordinary resolver precedence. The run cannot loop forever.

Restart and resume

At human waiting, all authoritative state is committed and no subscriber is retained. A new application process uses the same database. The decision service loads the work item and cursor, commits the decision, and invokes the stored mode. The same run continues rather than creating a replacement run.

Figure 3 gives the persistent lifecycle.

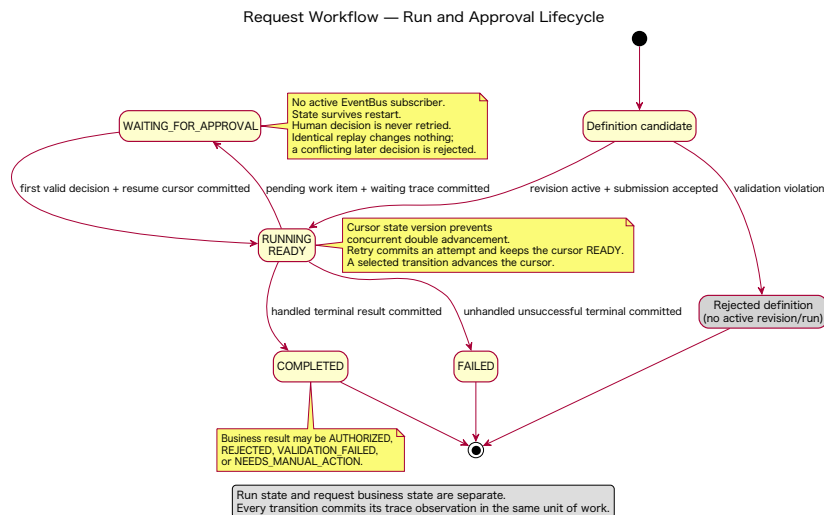


Figure 3. Workflow run and cursor lifecycle.

Architecture

Modular-monolith rationale

One modular process is appropriate because the academic comparison concerns control style, not network distribution. Splitting the product into services would add deployment, contracts, failure modes, and broker operations without creating stronger evidence for the implemented business rules. Layer and module boundaries still separate presentation, application, domain, and persistence responsibilities.

The principal trade-off is enforcement. Process boundaries cannot enforce module ownership, so source organization, ports, tests, and review must prevent routes or infrastructure adapters from owning domain policy. For the bounded project, this cost is lower than operating a distributed topology.

Persistence and transaction boundaries

Figure 4 shows conceptual ownership. Workflow drafts are mutable; revisions are immutable. A run points to one revision and one Purchase Request. Cursor, attempts, approval records, effects, and trace belong to the run/request context, never to the reusable definition.

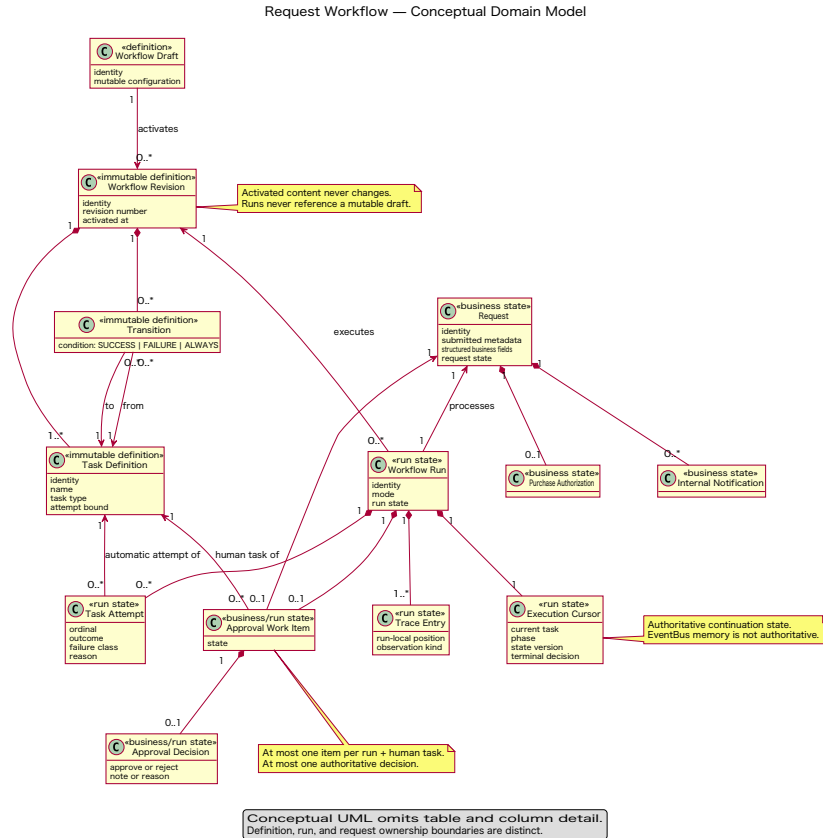


Figure 4. Conceptual domain and persistence ownership.

Each observable step commits related state and trace together. Submission commits request, run, cursor, and initial trace. An automatic step commits attempt, effect, cursor, and observations. Entering approval commits work item and waiting state. A decision commits the authoritative choice and resumed cursor. This design reduces partial-state explanations and supports recovery from the cursor.

EventBus semantics

The EventBus is a small synchronous infrastructure adapter. The choreographer registers a callback by run key, publishes AdvanceRun, and removes the handler when the active processing scope ends. Event data carries stable identity/version information; it is not a copy of authoritative business state. Committed SQLite state is read for each advancement.

This provides event-reaction structure suitable for comparing choreography with orchestration inside one process. It does not provide a queue, persistence, delivery acknowledgment, replay, consumer groups, backpressure, independent deployment, or distributed transaction semantics.

Deployment

Figure 5 shows the only supported topology: browser, one FastAPI application, and SQLite. Local startup and Compose use the same boundary. Compose adds a named volume for the database, not additional services.

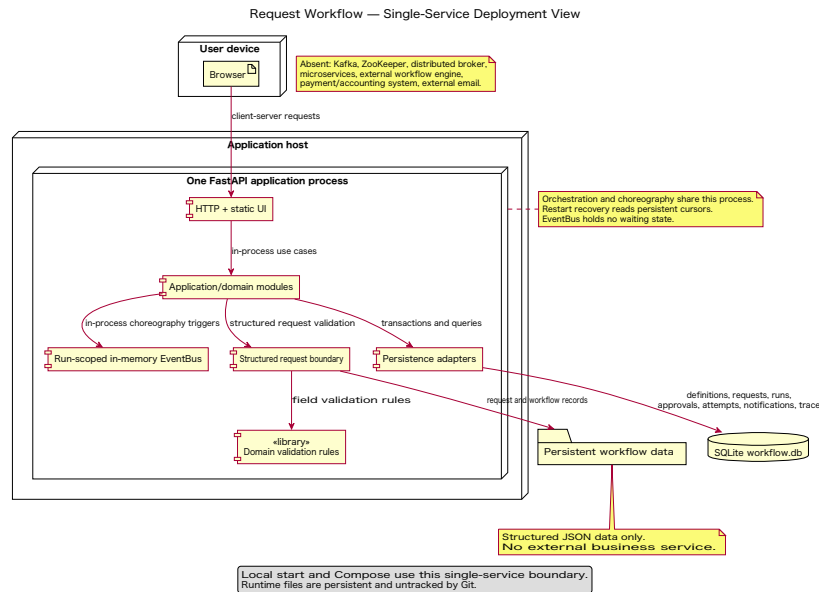


Figure 5. Optional single-service deployment.

ADR summary

Nine ADRs record the modular monolith, definition/revision/run ownership, shared result/retry/resolver contract, orchestration waiting, transient EventBus choreography, persistent cursor/atomic steps, deterministic fault adapter, structured request boundary, and closed designer. ADR status means implemented project decision; it does not imply separate professor approval.

Workflow Designer

The designer proves that the workflow core is reusable beyond one hard-coded sequence while keeping configuration safe. A process owner can create a draft, provide a workflow name, define tasks with stable keys and display names, select one start task, choose from four task types, set positive attempt bounds for automatic tasks, and define directed transitions using SUCCESS, FAILURE, or ALWAYS.

Validation rejects zero or multiple starts, unknown task types, invalid keys, missing references, self-edges, cycles, unreachable tasks, invalid terminals, ambiguous equal-precedence edges, missing/invalid automatic attempt bounds, and attempt bounds on human approval. Issues are returned before activation. Activation snapshots accepted content into an immutable revision. Runs reference the snapshot rather than the mutable draft.

The designer is not a general low-code product. It has no drag-and-drop BPMN parity, arbitrary scripts, expressions, plugins, user-supplied executors, nested workflows, cycles, timers, parallel gateways, compensation, or service discovery. The closed boundary is an engineering feature: it makes every executable type known to validation and the executor registry.

Custom algorithms and invariants

Algorithm	Purpose / input / output	Invariant and failure behavior	Source and tests	UML
Graph validation	Input draft/ revision graph; output ordered issues or acceptance	Exactly one start, reachable acyclic graph, valid tasks/ edges; invalid input never activates	app/domain/ validation.py, domain/constructor tests	Figure 6
Transition resolver	Input task result and outgoing edges; output selected edge or terminal	Specific outcome precedes ALWAYS; ambiguity rejected earlier	app/domain/ resolver.py, resolver tests	Figure 6
Bounded retry	Input task type/ result/count/ bound; output retry yes/no	Only retryable automatic failure below bound retries; no edge during retry	app/domain/ retry.py, step/ retry tests	Figure 6
Orchestration	Input run identity/cursor; output waiting or terminal projection	Central loop uses committed shared steps; stops at wait/ terminal	app/application/ orchestration.py, orchestration tests	Figure 7
Choreography	Input run trigger/ version; output waiting or terminal projection	One run-scoped handler; cursor authoritative; handler removed	app/application/ choreography.py, EventBus/ choreography tests	Figure 8
Persistent wait/ resume	Input human task/decision; output work item then resumed cursor	No open request/ subscriber while waiting; same run resumes	app/application/ approval.py, approval/restart tests	Figures 3, 7, 8
Decision idempotency/ conflict	Input work item and choice; output existing/ new result or conflict	One authoritative choice; identical replay cannot advance twice	approval domain/ service, approval tests	Figure 4
Run/revision isolation	Input run creation/step; output run-owned records	Run keeps one immutable revision; records never cross run identity	constructor/ persistence services and tests	Figure 4

Figure 6 makes the retry-before-routing order explicit.

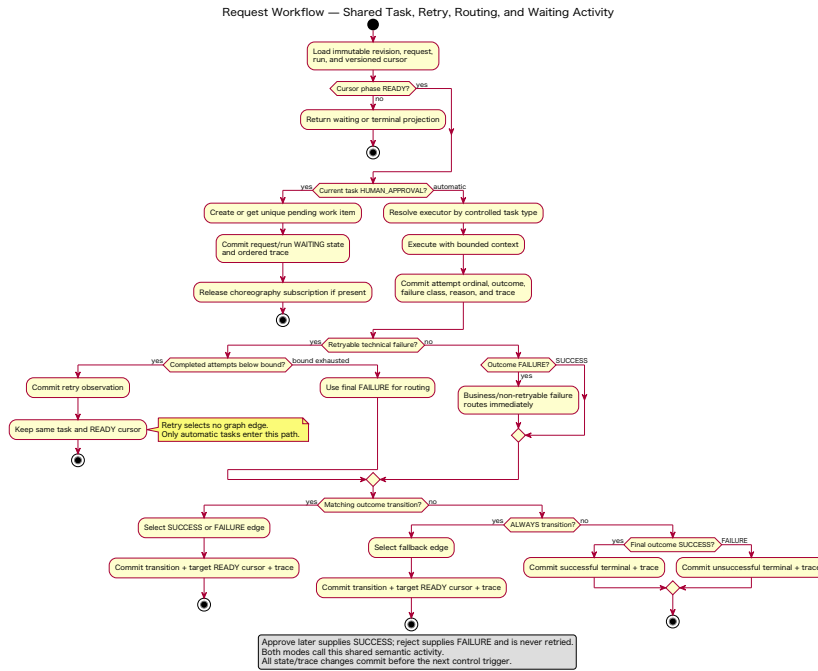


Figure 6. Graph execution, retry, routing, and human-wait activity.

Graph validation

Validation protects runtime assumptions before persistence exposes an active revision. It normalizes task types and conditions, checks structural constraints, computes reachability, detects directed cycles, and checks outgoing transition precedence. The main path returns no issues and activation may proceed. The failure path returns deterministic issues; no run or active revision is created from the rejected specification.

Deterministic resolver

The resolver is intentionally small and independent of Purchase Request semantics. Given a task result and outgoing transitions, it selects the unique matching outcome-specific edge. If absent, it selects a unique ALWAYS edge. If no eligible edge exists, it returns successful or unsuccessful terminal according to the final result. Retry and business effects happen outside the resolver, preventing coordination modes from acquiring different routing rules.

Bounded retry

Retry receives task type, failure classification, completed-attempt count, and configured maximum attempts. Human work is never retried automatically. Business and non-retryable failures route immediately. A retryable technical failure repeats only when another attempt remains. Exhaustion returns control to normal failure routing. The invariant is finite execution for each automatic task under the configured bound.

Persistent approval

Reaching human approval does not create an automatic attempt. The service creates or retrieves the work item, records waiting state, and commits. Decision validation distinguishes approval note and rejection reason. An identical replay returns the established decision result. A different choice conflicts. After commit, the cursor is ready and the coordinator re-enters the stored strategy.

Isolation and revision consistency

Every runtime query and mutation carries run identity; relevant uniqueness rules include run/task keys. A run's revision identifier never changes. Therefore later draft edits can create a new revision without changing the meaning of old attempts or traces. This is essential for audit interpretation.

Orchestration and choreography comparison

Dimension	Orchestration	Choreography
Control owner	Central loop	Temporary run-scoped event handler
Trigger	Direct strategy progression	Synchronous AdvanceRun publication
Business semantics	Shared services/rules/effects	Same shared services/rules/effects
Durable state	SQLite cursor and records	Same SQLite cursor and records
Human wait	Loop returns	Handler scope ends
Resume	Coordinator calls orchestrator	Coordinator creates a new handler scope
Strength	Straightforward control and debugging	Explicit event-reaction decoupling of advancement
Main risk	Central coordinator can accumulate responsibility	Handler lifecycle/version mistakes can cause leaks or stale work
Not claimed	Distributed scheduler	Durable/distributed messaging

Figure 7 shows central control. The HTTP request finishes when the run waits; approval later restarts the loop from persisted state.

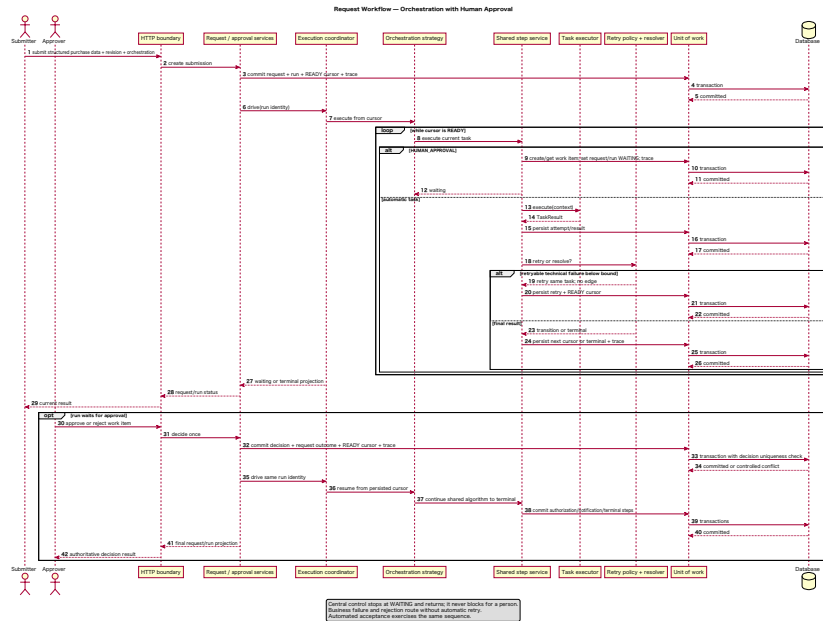


Figure 7. Orchestration sequence with persistent human wait.

Figure 8 shows the event-reaction variant. Commit precedes the next trigger, and no handler survives the waiting period.

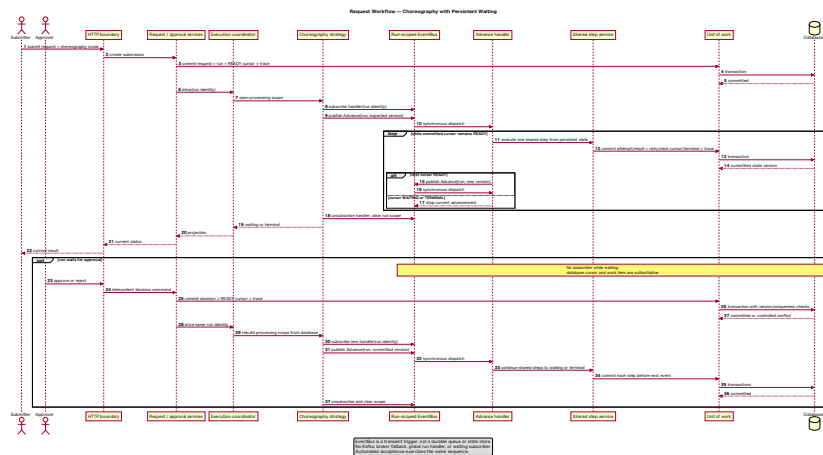


Figure 8. Choreography sequence with transient run-scoped EventBus.

The exact comparison claim is semantic parity for the repository's reference workflows, inputs, decisions, and deterministic fault schedules. It is not a general proof that orchestration and choreography are interchangeable in distributed systems.

Verification and evidence

Evidence classification

Claim class	Meaning in this report	Examples
Implemented and tested	Executed automated evidence asserts behavior	resolver, retry, approval, paired modes, restart
Implemented and inspected	Source/configuration directly shows structure	API paths, task catalog, SQLite models
Manually verified	A human/agent inspection was performed	report render, headings, current UI labels
Documented limitation	Deliberately absent or unverified	authentication, distribution, performance
Historical decision	Earlier direction preserved for evolution	generic prototype, Invoice/PDF increment
Professor-approved direction	Explicitly supplied project fact	WMS direction and two complex functionalities
Final design decision not separately approved	Implemented choice without approval claim	Purchase Request reference scenario, detailed ADR choices

Test categories and result

Category	Representative coverage	Result
Domain	Purchase Request fields, approval input, types, graph, resolver, retry	Included in 98 passed
Application	step service, constructor, orchestration, choreography, decisions	Included in 98 passed
Infrastructure	synchronous EventBus behavior and handler lifecycle	Included in 98 passed
Persistence	drafts/revisions, cursor, attempts, effects, conflicts, isolation	Included in 98 passed
Presentation	constructor API and connected Purchase Request journeys	Included in 98 passed
Deployment	real Uvicorn restart and deployment inventory	Included in 98 passed
Quality	dependencies and repository constraints	Included in 98 passed
Total	Cache-suppressed unchanged repository suite	98 passed

The authoritative final command is:

```
PYTHONDONTWRITEBYTECODE=1 python -m pytest -p no:cacheprovider -q
```

The exact result is reported only after executing the unchanged suite. Test quantity is not used as a proxy for quality; the business matrix, negative cases, restart behavior, and source boundaries explain what the number contains.

Restart evidence

The restart tests use a temporary isolated SQLite database and real Unicorn subprocesses rather than reconstructing application objects in one test process. Each test starts the server, submits a request, verifies waiting work, stops the process, launches a new server against the same database, submits the decision, and verifies that the same run reaches AUTHORIZED. One test uses orchestration and one choreography. This is recovery evidence for the local architecture, not high-availability evidence.

API and OpenAPI evidence

OpenAPI inspection confirms 12 current paths including `/api/requests`, `approval collection/detail/decision`, `run status`, and `workflow draft/validation/activation/revision` endpoints. There is no legacy `/api/v3` runtime. Connected presentation tests exercise JSON payloads through the FastAPI boundary rather than only calling domain functions.

UI evidence

Source and local smoke inspection confirm the exact headings **1. Submit Request**, **2. Approver Inbox**, **3. Run Status / History**, and **4. Workflow Designer**, plus collapsed **Demonstration Controls**. The final reconciliation did not invent usability study results. New automated screenshots could not be captured through the isolated in-app-browser network while keeping the unauthenticated local API bound to loopback; therefore the report relies on current HTML/API evidence and does not present synthetic screenshots as manual evidence.

Docker and static verification

`docker compose config --quiet` verifies Compose syntax and interpolation for the single service. It does not prove a production deployment. JavaScript syntax is checked with Node, `pip check` verifies installed dependency consistency, `git diff --check` detects whitespace errors, link/path validation checks repository references, and terminology search identifies obsolete active-language risks. These are supporting quality checks rather than business scenario tests.

Quality evaluation

Correctness

Correctness is supported by deterministic domain tests, business journeys, negative cases, and the shared semantic path. The most important design choice is that both modes call the same step service, resolver, retry policy, approval service, and persistence operations. This reduces the space in which parity can drift.

Reliability and recovery

Bounded attempts prevent infinite automatic retry. Persistent work avoids holding runtime resources while a person decides. State-version conflicts protect against stale advancement. Identical decision replay is idempotent. Real-process restart tests demonstrate recovery at the most important long-lived boundary. Reliability remains local to one application and SQLite; no redundant node or database failover exists.

Usability

Available usability evidence is structural, not experimental. The page leads with the product name, structured request, pending work, and business result. Technical trace and failure controls are secondary/collapsed. Meaningful request context appears in approval work. No user interviews, task-completion measurements, accessibility audit, or comparative usability study were conducted, so none is claimed.

Maintainability

Domain algorithms are small and testable without HTTP or database dependencies. Application services coordinate ports and controlled effects. Infrastructure implements synchronous events and persistence. A closed enum-based task catalog makes executor completeness checkable. Immutable revisions protect historical interpretation. The main maintainability risk is the size of persistence/application coordination required for atomic state and trace; focused tests and explicit ownership mitigate it.

Security

Input validation and closed task types reduce accidental misuse, but the application has no authentication, RBAC, CSRF strategy, secret management design, rate limiting, audit access control, security headers review, penetration testing, or threat-model validation. It must not be exposed as a production service without substantial work.

Scalability

No scalability claim is made. SQLite, synchronous EventBus dispatch, and one FastAPI service fit demonstration and automated testing. There are no load-test numbers, throughput measurements, queue-depth observations, or horizontal scaling experiments. A future distributed design would require different event durability, idempotency, partitioning, and operational evidence.

Reuse disclosure

FastAPI provides HTTP routing and dependency injection; Uvicorn provides the ASGI server; Pydantic validates transport shapes; SQLAlchemy maps persistence; SQLite stores local state; pytest and HTTPX support tests; Docker/Compose provide optional packaging; PlantUML renders diagrams; Pandoc and WeasyPrint build this report. These tools are general-purpose dependencies.

They do not provide this project's graph rules, transition precedence, retry policy, cursor lifecycle, work-item semantics, decision conflict behavior, revision model, run isolation, audit vocabulary, scenario matrix, or two control strategies.

Camunda, n8n, and Temporal were behavioral references only; no external workflow engine, product workflow, UI, or source code is embedded. The full disclosure is maintained in `docs/REUSE_DISCLOSURE.md`.

Limitations and future work

Current limitation	Consequence	Credible future work
No authentication/RBAC	Logical actors are not access-controlled	Identity integration, role policies, authorization tests
Internal authorization only	No order or budget operation occurs	Explicit procurement adapter and transactional boundary
Internal notification only	No external delivery	Outbox and email/provider adapter with delivery evidence
Synchronous in-process EventBus	No durable/distributed choreography	Durable broker only if multi-service need is justified
SQLite single service	Limited operational topology	Production database, migrations, backup/recovery plan
Conditional DAG only	No parallel/cyclic/compensation patterns	Formal semantics and new validator/runtime design
Closed designer	Not a general low-code platform	Carefully add bounded configuration, not arbitrary code
No measured usability/accessibility	UI quality is structurally inspected only	Task study and WCAG-focused audit
No security testing	Unsafe for public production exposure	Threat model, auth, hardening, dependency and penetration review
No load/performance evidence	Scalability is unknown	Workload model, benchmarks, observability, capacity targets
Deterministic examination faults	Not representative of real integrations	Adapter contract tests and controlled integration sandbox
Old Invoice data not migrated	Previous development DB is incompatible	Migration only if historical data becomes a real requirement

Future work should start with identity/security and requirements for real organizational integration, not with adding visually impressive but unsupported workflow features.

Conclusion

The final Event-Driven Workflow Management System turns a technical workflow core into an understandable bounded product. A requester supplies structured purchase information, a manager receives persistent work, and the system resumes the same run to produce an internal authorization or controlled negative result with ordered evidence. Graph validation, deterministic routing, bounded retry, immutable revisions, run isolation, idempotent decisions, and restart recovery make the behavior explainable across failure and time.

Orchestration and choreography are compared fairly because business semantics and persistence are shared. The orchestrator centralizes advancement; the choreographer reacts through a temporary synchronous run-scoped handler. The report does not extend that result to distributed systems.

The project remains appropriately limited for one student and approximately 100 hours. Its strongest evidence is not feature breadth but correspondence between problem, requirements, custom logic, scenarios, tests, architecture, and stated limitations. Final academic acceptance remains the professor's decision.

References

1. FastAPI documentation, framework concepts used for HTTP presentation.
2. SQLAlchemy documentation, ORM and transaction concepts used for persistence.
3. SQLite documentation, local relational database behavior.
4. pytest documentation, automated test execution.
5. PlantUML documentation, architecture diagram rendering.
6. Docker Compose specification, optional single-service configuration.
7. Project repository: requirements, ADRs, CR-003, source modules, tests, and evidence register.
8. Camunda, n8n, and Temporal public product concepts, consulted only as behavioral references as disclosed in the reuse record.

Appendix A - Requirement-to-implementation/test traceability

Requirement group	Implementation responsibility	Test/evidence group
Graph and routing	domain/validation.py, domain/resolver.py	validation/resolver tests
Retry and attempts	domain/retry.py, application/ step_service.py	retry/step-service/persistence tests
Structured request	domain/ purchase_request.py, submission/validation services	domain and connected journey tests
Human decision	approval domain/application/ persistence	approval, journey, conflict, restart tests
Immutable configuration	constructor service and persistence	constructor domain/API/ persistence tests
Orchestration	execution coordinator/ orchestrator	orchestration and paired journey tests
Choreography	choreographer and in-memory EventBus	choreography/EventBus/parity tests
Recovery/isolation	cursor/version persistence and coordinator	restart and isolation tests
Presentation	request, approval, run, constructor APIs and static UI	presentation and UI-structure tests
Deployment	launcher, Dockerfile, Compose	deployment inventory and configuration checks

Appendix B - API summary

Method/path	Purpose
POST /api/requests	Submit structured Purchase Request data and start execution
GET /api/approvals	List pending approval work
GET /api/approvals/{work_item_id}	Read one work item and request context
POST /api/approvals/{work_item_id}/decision	Commit approve/reject and resume the run
GET /api/runs/{run_id}	Read business/run state, effects, attempts, and trace
GET /api/workflows/drafts	List drafts
POST /api/workflows/drafts	Create a bounded draft
GET/PUT/DELETE /api/workflows/drafts/{draft_id}	Read, update, or delete a draft
POST /api/workflows/validate	Validate a supplied draft projection
POST /api/workflows/drafts/{draft_id}/validate	Validate a stored draft
POST /api/workflows/drafts/{draft_id}/activate	Create an immutable revision
GET /api/workflows/revisions/{revision_id}	Read an immutable revision

Appendix C - Defense evidence checklist

1. State the problem and system boundary before showing execution details.
2. Submit a valid structured request in orchestration mode.
3. Show persistent approval context and explain that the initial request returned.
4. Approve and show AUTHORIZED, authorization identifier, notification, and trace.
5. Demonstrate retry-then-success or show exact retry evidence.
6. Show the bounded Workflow Designer and immutable activation.
7. Compare Figures 7 and 8 while emphasizing shared semantics.
8. Show 98 passed, including the two real-process restart tests.
9. Show one-service Compose/deployment evidence and the absence of a broker.
10. End with limitations; never imply ordering, payment, distributed choreography, production security, performance, or separate professor approval of the reference scenario.

Appendix D - Final claim ledger

Claim	Classification	Evidence/boundary
WMS direction and two complex functionalities were approved	Professor-approved direction	Project brief supplied for final reconciliation
Purchase Request is the reference workflow	Final design decision not separately approved	Current implementation and CR-003
Four task types and immutable revisions exist	Implemented and tested	source, constructor tests
Retry precedes routing and is bounded	Implemented and tested	domain/application tests and Figure 6
Both modes preserve repository scenario semantics	Implemented and tested	paired normalized tests
Waiting survives real process recreation	Implemented and tested	two Uvicorn restart tests
API has current request/ approval/run/workflow paths	Implemented and inspected	OpenAPI and routes
UI has four task-oriented views	Implemented and inspected	static UI and local smoke evidence
Single-service Docker topology is valid	Implemented and inspected	Compose config and inventory tests
Report layout is readable	Manually verified	post-build page rendering and visual QA
Product is production-secure or scalable	Not claimed	documented limitation; no supporting evidence